Stability Threshold and Performance Degradation Analysis of Priority-Aware SDN in Healthcare IoT Systems

A Vibhas Reddy, S Pavan Kumar, Thangam S
Department of Computer Science and Engineering
Amrita School of Computing, Bengaluru
Amrita Vishwa Vidyapeetham, India
bl.en.u4cse23004@bl.students.amrita.edu, bl.en.u4cse23051@bl.students.amrita.edu,
s_thangam@blr.amrita.eduedu

Abstract*—The rise of Internet of Things (IoT) devices in healthcare settings brings strict quality-of-service (QoS) needs for different traffic types, from critical emergency alarms to regular telemetry. Static network designs struggle to meet specific latency and loss targets because they cannot change scheduling decisions during operation. This paper introduces a priority-aware Software-Defined Networking (SDN) framework for healthcare IoT. It combines a Dijkstra-based dynamic path selection controller with a tabular Q-learning reinforcement learning (RL) agent. This setup actively improves weighted-priority queue assignments while in use. The system is tested through a discrete-event simulation based on real ICU sensor data, which includes patient monitoring (ECG, infusion pumps, blood pressure) and environmental monitoring (smoke, fire, CO gas). These workloads are organized into four traffic classes: Emergency, Critical, Clinical, and Routine. The results show that the RL-enhanced SDN controller cuts overall P95 latency by 40% (13.1 ms compared to 21.8 ms) and reduces jitter by 84% (2.01 ms versus 12.32 ms) compared to static priority-weighted SDN. It also maintains 100% on-time delivery for Critical traffic with a degradation index of 0.276 against 0.302. These improvements come from the agent's ability to adaptively focus on significant class separation weights (E:52, C:51, L:50, R:1), which effectively isolates routine traffic during high network load. The proposed setup shows stable performance across all tested load situations, proving that reinforcement-learning-based SDN can be a strong scheduling solution for latency-sensitive healthcare IoT applications.

***Index Terms*—Software-Defined Networking, Healthcare IoT, Quality of Service, Reinforcement Learning, Traffic Prioritization, Network Scheduling**

I. Introduction

Data is as important as medicine to hospitals these days. Each ECG, infusion pump, pulse oximeter, and smoke alarm silently transmits data over the network, instantly adding to electronic health records and clinical decision support systems. However, in a hospital network, traffic cannot be treated in the same way as a standard office network, a delay that in this case can cost a life. Some signals are more urgent than others: a sensor failure or a smoke alarm must alert someone within milliseconds if not a couple of seconds, whereas a blood glucose reading can wait a bit longer without hurting. The true challenge, however, is not only in moving the data, but in knowing which data is important in the moment, and routing it.

This is how it has never been built in a conventional network. Traditional routing protocols (distance vector, link state) are based on static criteria such as the number of hops between nodes and do not take into account whether a packet has a life-critical purpose or not. No one looks at the other nodes, each one makes its own decision. This blindness becomes real damage under heavy loading, with increasing latency, jitter, and dropped packets, and most of all, affecting time-sensitive medical traffic. Software-Defined Networking (SDN) is there to save the day. When SDN separates the control plane from the data plane, a single controller has a bird's-eye view of the network: all links, all flows, all bottlenecks can be seen and calculated, and the best paths (with priorities) are pushed to every switch.

While SDN is already established for data centers and enterprise environments, healthcare IoT is a different game [1], and opportunities are still far from being fully explored here. The first issue that arises is the fixed priority weights. They are fine on a regular day, but hospitals don't have normal days — patients get sicker, machines break, emergencies arise and everything shifts. The static scheduler cannot adapt, if configured it will either deny bandwidth to critical traffic or allow low priorities to choke the pipes.

Reinforcement learning is the natural solution. In contrast, an RL agent monitors the health of the network's QoS, adapts the scheduling weights according to its observation, and learns, via trial, reward and repetition, a new policy that maintains every traffic class within the limits. The state space is small, and, with a few classes and thresholds, tabular Q-learning fits naturally in this context with no burden of deep learning. Following this, this paper proposes a priority-aware SDN architecture, which combines a Dijkstra based path-selection controller with a Q-learning agent that continuously tunes the weights of the queues, using a real set of sensor traces from the ICU with four different traffic classes, each with different

requirements on latency, jitter, and loss.

The principal contributions of this work are as follows:

- Priority-aware SDN controller using Dijkstra for dynamic path selection based on cost functions per-class that take into account real-time link utilisation and queue priorities.
- Tabular Q-learning agent with a compressed state space of 81 states and a designed reward function to simultaneously optimise latency, packet delivery on-time ratio, and packet loss among all classes of traffic.
- Discrete event simulation system using real ICU IoT sensor data traces, ensuring that experiments are repeatable under clinically motivated traffic scenarios and varying loads.
- Experimental results showing that with the help of RL, the SDN network performs 40% better on P95 latency, 84% better on jitter, and shows a smaller degradation index compared to priority-weighted static SDN, irrespective of load conditions.

II. Related Works

Tam et al. [2] stated that priority-aware resource management for adaptive service function chaining (SFC) enhances the performance of real-time intelligent IoT services in SDN/NFV-enabled environments. The study proposed an embedded deep reinforcement learning-based orchestration approach that dynamically configures virtual network functions (VNFs) according to IoT service priorities, thereby reducing transmission delays and improving resource utilization, delivery ratio, and throughput compared to existing approaches.

Su et al. [3] proposed QCORS (congestion-aware OpenFlow routing scheme), which can predict the link congestion and reroute the flow, thereby minimizing P2P delay while satisfying the QoS requirements.

Rostami and Goli-Bidgoli [4] reviewed that the load balancing methods under QoS considerations in SDN-based IoT networks were reviewed and found that the intelligent traffic and load management increases the network performance.

Asingwire et al. [5] stated that prioritized scheduling schemes in IoT-based healthcare systems improve the performance of heterogeneous delay-sensitive multi-server queuing environments compared to traditional FCFS and SJF approaches. The study proposed an analytical priority-based model that differentiates services according to packet delay sensitivity and packet size, enabling delay-sensitive and shorter packets to receive faster processing, thereby reducing mean slowdown and improving overall healthcare communication efficiency under overloaded network conditions.

Beitelspacher et al. [6] stated that sensor-driven priority routing in IoT healthcare networks significantly improves the timely delivery of critical medical data under congested network conditions. The study proposed a healthcare-aware routing mechanism that embeds healthcare identifiers into IP packet headers and modifies router QoS policies to prioritize medical data packets, resulting in nearly 80% lower latency compared to conventional routing approaches, thereby enhancing remote healthcare reliability and reducing risks to patient safety.

Mannem et al. [7] stated that IoT-based SDN smart health monitoring systems improve healthcare service delivery by enabling secure, continuous, and real-time patient monitoring through sensors, wireless sensor networks, and cloud integration. The study highlighted that Software-Defined Networking (SDN) enhances the security, privacy, and reliability of healthcare data transmission by efficiently managing network traffic and safeguarding sensitive medical records, thereby supporting rapid treatment and dependable healthcare services for senior citizens and critically ill patients.

Parveen and Bhuvaneswari [8] proposed NF-MTAR is a neuro-fuzzy multi-topology adaptive routing technique using traffic, resource, energy, and link quality metrics to enhance the throughput, delay, energy efficiency, and reliability in healthcare IoT.

Hosseinzadeh et al. [9] proposed QQMR is a multipath routing scheme for IoMT/WBANs based on Q-learning to classify medical data based on their priorities and fuzzy c-means clustering, which improves delivery ratio and minimizes delay and energy consumption.

Pathan et al. [10] proposed a MILP formulation with two greedy heuristics (PEMA and PEDL) for priority, energy, and load aware routing in SDN datacenters, enhancing the flow success probability, energy saving, and load balancing.

Dilek et al. [11] stated that QoS-aware IoT networks and communication protocols are essential for ensuring reliable and efficient service delivery in increasingly complex IoT environments with diverse application requirements. The study presented a comprehensive survey of QoS support mechanisms in IoT architectures, communication protocols, QoS metrics, and resource optimization techniques, while also identifying the limitations of existing approaches and highlighting future research directions for achieving end-to-end QoS provisioning in IoT systems.

Thangam et al. [12] conducted research into MPLS and its application to increase QoS in 5G networks through the creation of Label Switched Paths (LSPs) for various traffic categories (eMBB, URLLC, and mMTC). The authors have proven that MPLS is an effective tool for increasing the quality of communications in 5G networks using traffic engineering and routing via Python Scapy simulations.

Lakshmi Priya et al. [12] suggested an algorithm for optimized placement of SDN controller, employing Ryu Controller and Mininet that helps to decrease network latency and provide load balancing in campus networks. This algorithm provided improved scalability and fault tolerance with up to 35% reduction in network latency compared to traditional approaches.

Thangam et al. [13] developed the concept of Adaptive QoS for VoIP based on the algorithms of Call Sequencing, Adaptive Jitter Buffer, and Silence Suppression. This system dynamically improved network performance through latency reduction, jitter reduction, and packet loss reduction, which made real-time voice communication more efficient.

The reviewed studies collectively highlight the importance of priority-aware and QoS-driven resource management techniques in SDN-enabled IoT and healthcare networks. Various approaches such as deep reinforcement learning, Q-learning, neuro-fuzzy routing, congestion-aware routing, load balancing, and priority-based scheduling were proposed to improve network performance, reduce latency, enhance throughput, optimize energy consumption, and ensure reliable healthcare data transmission. These studies also demonstrate that integrating SDN, IoT, cloud computing, and intelligent routing mechanisms enables efficient traffic management, secure communication, and adaptive service delivery for critical healthcare and smart network applications.

III. Methodology

A. IoT Dataset

The dataset has been built by creating a simulated ICU environment with two beds with nine patient monitoring sensors and one bed control unit per bed that have been emulated via IoT-Flock. In the simulated environment, two traffic streams were observed – the first one is patient monitoring activities, and the second one is environmental conditions. These two streams form the basis for generating traffic in this study by controlling the inter-arrival time of packets, their size, and distribution of sensor topics.

The first patient dataset includes packets transmitted from the nine patient monitoring devices assigned for every ICU bed. Each of those packets contains four important pieces of information – the time of capture, the packet size in bytes, the MQTT sensor topic ID, and the payload of the packet. Eight sensor types provide data for this dataset – ECG monitors, infusion pumps, blood pressure monitors, EMG sensors, pulse oximeters, airflow monitors, glucometers, and temperature probes – all together represent the ongoing physiological monitoring of ICU patients. Among these, the AirFlow monitor is classified as Emergency traffic, given its direct relevance to respiratory distress conditions.

The second environment dataset includes MQTT messages coming from the virtual environmental monitoring system of the ICU. Multiple measurements from humidity sensors, barometers, solar radiation sensors, CoGas sensors, smoke detectors, and fire alarms can be found. Most sensors are working in low-priority conditions and therefore are classified as Routine traffic devices. However, the smoke and fire sensors are an exception - any MQTT message coming from them becomes Emergency traffic immediately.

Processing of both datasets takes place before the simulation procedure begins. Packet records are first parsed and their sizes are converted to kilobytes with a minimum size of 0.1 KB imposed in case of malformed data entries. Further, a traffic class is allocated to each individual packet depending on the MQTT topic name extracted from the packet content, as shown in Table I. If no topic name is available or null values appear in packet entries, they are filtered out of the dataset. Afterward, both streams are joined and sorted by their timestamp values in ascending order, thus forming a unified stream of packets. It is then replayed starting from the beginning of the list at the start of each trial.

TABLE I
MQTT Topic to Traffic Class Mapping

Sensor Topic	Dataset	Traffic Class
AirFlow	Patient	Emergency
Smoke	Environment	Emergency
Fire	Environment	Emergency
ECG	Patient	Critical
InfusionPump	Patient	Critical
BloodPressure	Patient	Critical
EMG	Patient	Clinical
PulseOximeter	Patient	Clinical
Glucometer	Patient	Routine
Temperature	Patient	Routine
Humidity	Environment	Routine
Barometer	Environment	Routine
SolarRad	Environment	Routine
CoGas	Environment	Routine

B. Healthcare IoT Traffic Classification Model

Healthcare IoT communications have intrinsically distinct QoS demands according to the urgency of their source devices. In order to implement differentiated forwarding assurances for this diverse set of sensor data transmissions, all traffic flows are classified into one of $K = 4$ priority groups as per their medical severity level:

$$C = \{c_{\text{emrg}},\ c_{\text{crit}},\ c_{\text{clin}},\ c_{\text{rtn}}\}$$

Each class $c \in C$ is characterised by a QoS constraint tuple:

$$\text{QoS}(c) = (D_c,\ L_c,\ J_c)$$

where D_c denotes the maximum tolerable end-to-end latency (ms), L_c denotes the maximum tolerable packet loss rate (%), and J_c denotes the maximum tolerable jitter (ms). An initial scheduling priority weight w_c is also assigned to each class to seed the weighted-priority queue scheduler. The full QoS constraint matrix is presented in Table II.

C. Traffic Generation

Traffic injection occurs through a sliding window replay of the IoT traffic data set, with all empirical packets belonging to the current window obtained every step of simulation of duration $\Delta t = 25$ ms. For each packet, the number of duplicates that are injected into the network is obtained from a Poisson distribution whose mean value is the offered load factor L:

$$\text{count} \sim \text{Poisson}(\lambda = L)$$

In this way, it ensures that both time and topic distributions are the same as those found in the original sensor data set but allows for continuous variations in traffic load ranging from sub-nominal values ($L = 0.5$) to overload ($L = 3.5$). In order to create realistic per-frame size variability, the size of each generated packet is taken from a normal distribution

TABLE II
QoS CONSTRAINT MATRIX

Class	Priority Weight w_c	Max Latency D_c (ms)	Max Loss L_c (%)	Max Jitter J_c (ms)	Example Devices
Emergency	10	30	0.1	10	AirFlow
Critical	5	80	1.0	25	ECG, SpO_2, Infusion pump
Clinical	3	150	3.0	55	BP monitor, Glucometer
Routine	1	300	8.0	120	Bed occupancy, Room climate

about the empirical packet size, with 10% standard deviation and a minimum of 0.1 KB, to account for coding differences and overheads. The source node for each packet is randomly chosen among those of the hospital devices corresponding to its traffic class: ICU and ward nodes generate Emergency and Critical packets, ward, laboratory, and outpatient nodes generate Clinical packets, and low-acuity nodes (pharmacy and radiology) generate Routine packets, with all packets destined for the central Electronic Health Record (EHR) server.

D. Hospital Network Topology and SDN Controller

The simulated network model follows a hierarchical hospital campus network topology with four layers [15]. Edge layer includes wards, ICU, laboratory, outpatient, pharmacy, and radiology sensors as individual nodes. These nodes have connections towards the aggregation layer that includes six edge switches, which are connected to the core layer including four high-speed switches. Topmost layer of the network includes the control and services layer containing SDN controller and EHR server as the universal receiver for all the sensor data traffic. All the links in the network are defined using their bandwidth C_e (kbps), propagation delay d_e (ms), and queue state for each class.

The routing process is conducted by the logically centralized SDN controller by applying Dijkstra's algorithm to the network graph weighted dynamically. The cost of the edge e for packets belonging to the traffic class c is:

$$C_{\text{SDN}}(e, c) = d_e + \frac{1000 \cdot Q_{\text{eff}}(c)}{C_e} + 18 \cdot U(e)$$

where $Q_{\text{eff}}(c)$ is the normalized queue occupancy considering the class priority weight w_c, and $U(e)$ is the current link utilization. Such implementation helps to push the high priority traffic through the least loaded paths. Paths are stored in a flow table which takes the parameters (source, destination, traffic class) as keys to avoid recalculation of the path for future packets of the flow. The packets will be routed hop-by-hop with delay accumulated at each edge and possibly getting lost if the queue capacity is reached. On-time deliveries will be determined by comparing the time when the packets reach the destination host with the deadline of the traffic class D_c.

E. Reinforcement Learning Agent for Adaptive Queue Weight Optimisation

Though fixed priority weightings suffice for reliable operation in stable traffic environments, they lack the capacity to react to dynamic shifts in terms of patient surge, sensor burst, and network congestion. In order to cope with this problem, an online learning Q-agent is implemented in order to constantly monitor per-class QoS metrics and modify the scheduler weight vector $\mathbf{w} = (w_{\text{emrg}},\ w_{\text{crit}},\ w_{\text{clin}},\ w_{\text{rtn}})$ online. It is noteworthy that the weights affect routing cost calculation, thus providing a feedback loop from network conditions to the forwarding decisions.

At each decision step, the agent receives the network state encoded as a four-dimensional vector, with each dimension corresponding to one of the traffic classes. The possible values of the dimensions are 0 (healthy condition), 1 (a warning condition), and 2 (a critical condition). They are computed on the basis of worst-case QoS ratio of latency, drop probability, and deadline miss ratio for each class. Thus, there exist $3^4 = 81$ different states that are enough to represent congestion scenarios of practical interest without causing memory blow-up.

Given the observed state, the agent chooses any one out of 17 different possible weight adjustment actions, which may vary from no operation and modest single-class adjustments to strong priority promotion actions and compound actions that promote high priority classes while decreasing lower priority classes. There is also an action that isolates the traffic under emergency conditions to give strong preference to Emergency traffic in extreme cases. Following each update, the weights are clamped to the range $[1, 50]$ and the strict clinical ordering $w_{\text{emrg}} > w_{\text{crit}} > w_{\text{clin}} > w_{\text{rtn}}$ is maintained.

The reward function of the agent is calculated on the basis of the following equation, which depends on all the classes:

$$R = \sum_{c \in C} \text{imp}_c \left(0.4 \cdot \text{lat_score}_c + 0.4 \cdot \text{on_time}_c - 0.2 \cdot \text{penalty}_c\right)$$

where lat_score is a measure of proximity of the $p95_c$ value to the deadline of the class, on_time is the ratio of the number of transmitted packets under the deadline, while the penalty value introduces a quadratic penalty on the packet drop and miss rates in order to penalize high QoS violations. The weights of the classes Emergency, Critical, Clinical and Routine are set to be equal to 4.0, 3.0, 2.0 and 1.0 respectively.

For updating the action values in the environment, the agent uses the following Q-learning update algorithm:

$$Q(s, a) \leftarrow (1 - \alpha)\, Q(s, a) + \alpha \Big(R + \gamma \max_{a'} Q(s', a') \Big)$$

with the parameters $\alpha = 0.20$, $\gamma = 0.90$. In the course of actions' selection, the ε-greedy approach with geometrical decaying of ε from 0.50 to 0.05 is used with the coefficient of decaying equal to 0.97 per step.

Every simulation trial starts with a warm-up period during the first 20% of simulation time. At this stage, the agent is free to explore and adapt without its actions influencing the recorded evaluation metrics. The actual evaluation is performed later, using the weights that have been only partially trained, with learning still going on. For experiments conducted at different load levels, the learned weight vector from one experiment is used as the initialization for the next one, enabling the experience gained in easier scenarios to help converge faster in harder scenarios. As a result of learning weights, the cost function for SDN routing becomes:

$$C_{\text{SDN+RL}}(e,c) = d_e + \frac{1000 \cdot Q_{\text{eff}}(c;\, \mathbf{w}^{\text{RL}})}{C_e} + 18 \cdot U(e)$$

where $\mathbf{w}^{\text{RL}}$ is the dynamically adjusted vector of weights.

F. Performance Metrics and Evaluation

The evaluation exhaustively scans the provided load multiplier L from 0.5 to 3.5 with increments of 0.2, resulting in 16 load points ranging from lightly loaded to heavily congested network scenarios. Each load point simulates 24 seconds of network emulation time with the granularity of $\Delta t = 25$ ms and assigns different seeds for random processes for every load point to provide consistent results without affecting each other between load points. There are two routing algorithms which are evaluated with identical topology, dataset of IoT devices, and load sweep parameters: static priority-weighted SDN routing (SDN) and reinforcement learning-based SDN routing (SDN+RL) to make sure the only difference is the routing algorithm itself.

At every load point for each traffic class, packet generation, packet delivery, packet dropping, and on-time packet delivery numbers are counted. Then, from the logs of all delivered packets, end-to-end packet latency $\bar{L}_c$, packet 95th percentile $p95_c$, and jitter J_c - defined as the population standard deviation of latencies of all packets - are calculated. Also, packet drop rate, packet deadline miss rate, packet loss rate - their total - and on-time delivery ratio are recorded for every traffic class.

To compare overall system QoS across load levels through a single scalar, the Weighted Performance Degradation Index $DI(L)$ is introduced:

$$DI(L) = \sum_{c \in C} w_c \cdot \min\left(10,\ \max\left(\frac{p95_c}{D_c},\ \frac{\text{drop}_c}{L_c},\ \frac{\text{miss}_c}{L_c}\right)\right)$$

For each class, the worst-performing QoS dimension is normalised against its target and capped at 10 to prevent a single failing class from distorting the index. Class importance weights of $w_{\text{emrg}} = 0.45$, $w_{\text{crit}} = 0.35$, $w_{\text{clin}} = 0.15$, and $w_{\text{rtn}} = 0.05$ — summing to unity — are applied, with Emergency and Critical traffic jointly accounting for 80% of the index. A $DI(L) = 1.0$ marks the exact QoS boundary: values below indicate spare capacity, while values above signal active QoS violations dominated by life-critical traffic.

Finally, the network stability threshold L^* is defined as the lowest load level at which QoS instability persists across two or more consecutive load points — a requirement that prevents transient spikes from being misclassified as structural degradation. A load point is declared unstable if any traffic class breaches any of the following: its 95th-percentile latency exceeds the class deadline by a tolerance multiplier ($\times$1.5 for Emergency and Critical, $\times$2.0 for Clinical, $\times$2.5 for Routine), its drop or deadline miss rate exceeds twice the permitted loss target, or its jitter exceeds twice the class jitter target. The progressively relaxed multipliers for lower-priority classes reflect the greater tolerance they can afford before instability is formally declared.

IV. Results and Analysis

This section provides a detailed analysis of the two algorithms - priority-weighted SDN routing algorithm (SDN) and SDN routing algorithm with the reinforcement learning module (SDN+RL) - for the whole range of offered load from $L = 0.5$ to $L = 3.5$. The performance metrics considered include the class-based latency, jitter, and Weighted Performance Degradation Index (WPDI).

A. Per-Class Latency Performance

A summary of the 95th percentile end-to-end latency for each traffic type at maximum load levels ($L = 3.5$), in which the differences between the two configurations are most evident, is shown in Table III.

TABLE III
PER-CLASS P95 LATENCY AT PEAK LOAD ($L = 3.5$)

Metric	**SDN+RL**	**SDN**
Emergency P95	13.0 ms	12.8 ms
Critical P95	14.0 ms	15.3 ms
Clinical P95	15.9 ms	32.1 ms
Routine P95	20.9 ms	69.4 ms
Overall P95	13.1 ms	21.8 ms
Overall Jitter	2.01 ms	12.32 ms
Critical Loss	0.00%	0.00%
Critical Delivery	100%	100%
Degradation Index	0.276	0.302
Queue Weights	E:52 C:51 L:50 R:1	E:10 C:5 L:3 R:1

Starting with the Emergency traffic class, the P95 latency in the SDN+RL configuration amounts to 13.0 ms, versus 12.8 ms in static SDN. This small difference of 0.2 ms is easily explained by the fact that the RL agent found a way to narrow the weight gap between the Emergency, Critical, and Clinical classes (reaching E:52, C:51, L:50, R:1) since it turned out that the network had enough bandwidth to guarantee Emergency deadlines without keeping too high priority gaps. The Critical traffic class shows some improvement in performance: the P95 latency dropped from 15.3 ms to 14.0 ms (a drop by 8.5%), which is well within the 80 ms deadline.

Most remarkable changes are observed for low-priority classes. Latency of Clinical class P95 decreases from 32.1 ms to 15.9 ms, or 50.5%, because of increased scheduling weight assigned by RL agent to this class compared to the fixed static hierarchy (10:5:3:1) when it led to queuing of clinical flows

behind high-priority flows of Emergency and Critical classes. In case of Routine class, the situation is the same as in the first example. P95 latency of this class drops from 69.4 ms to 20.9 ms. It is important to note that this result was not obtained due to increased priority of Routine class but due to balanced queue occupancy by increased weights of Clinical class.

When looking at the entire load sweep shown in Fig. 1, the difference is even more apparent. While static SDN demonstrates superlinear increase in latency, which goes up from 11.5 ms at $L = 0.5$ to 21.8 ms at $L = 3.5$ with a sharp increase after $L = 2.1$ due to the saturation of the weighted-priority queue, SDN+RL increases its latency from 10.5 ms to only 13.1 ms over the same range — showing almost linear behavior with 40% decrease in overall latency at maximum load compared to static SDN.

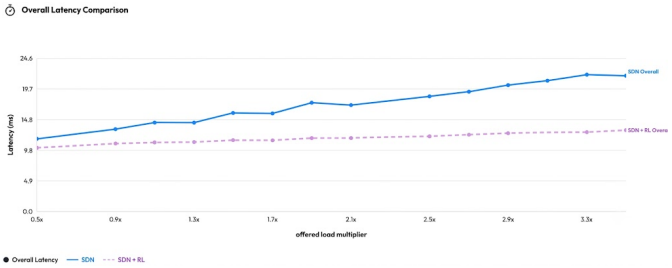


Fig. 1. Overall P95 latency comparison between SDN and SDN+RL across offered load levels.

B. Jitter Analysis

Figure 2 illustrates overall jitter for the full load sweep, highlighting one of the most notable differences between the two designs.

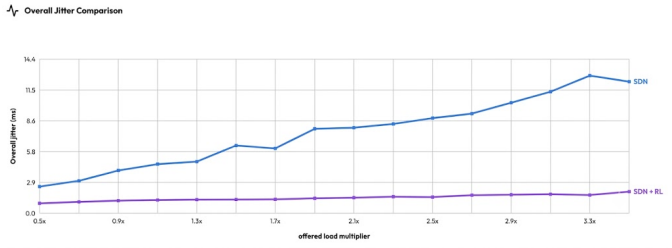


Fig. 2. Overall jitter comparison between SDN and SDN+RL across offered load levels.

For static SDN, jitter increases rapidly and monotonically from about 2.6 ms at $L = 0.5$ to 12.32 ms at $L = 3.5$, increasing slowly up to $L = 1.3$, staying constant for a while between $L = 1.5$–1.7, and increasing sharply thereafter starting from $L = 2.1$. Such a behavior is due to static priority queuing, where waiting time experienced by Clinical and Routine packets becomes increasingly variable once Emergency and Critical queues get saturated. For SDN+RL, on the other hand, jitter remains extremely stable, varying from 1.5 ms to 2.01 ms over the whole range of loads. This means that at maximum load jitter drops by 84% (from 12.32 ms to 2.01 ms).

This jitter pattern is the direct result of the equalized queue service rates provided by the converged RL scheduling scheme (E:52, C:51, L:50, R:1). Through even distribution of the service weights between the medically relevant traffic classes, the agent mitigates the burst-based increase in jitter observed with the static priority levels. As far as the clinical use case is concerned, this is of great importance: stream-based medical monitoring tasks like the measurement of ECG or SpO_2 depend on packet timing variations, whereby high jitter may distort the waveform and suppress alarms, as well as negatively affect the data quality at the EHR server.

C. Weighted Performance Degradation Index

Figure 3 shows the Weighted Performance Degradation Index $DI(L)$ for both schemes over the complete range of loads, while Table IV shows the class-based QoS ratio decomposition at the peak of the load.

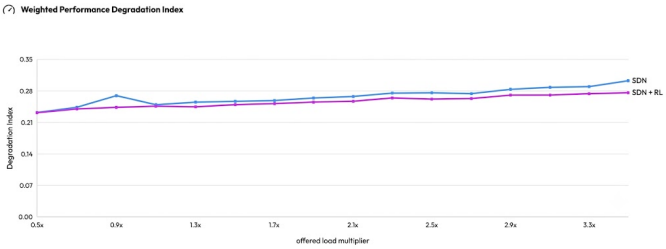


Fig. 3. Weighted Performance Degradation Index comparison between SDN and SDN+RL across offered load levels.

Over the entire range of loads, both schemes ensure $DI(L) < 1.0$ and thus do not violate their QoS constraints for any of the tested cases. This proves a critical benchmark since static SDN alone is enough to ensure QoS stability in the tested hospital environment. In other words, SDN+RL does not save the system from collapse; rather, it offers improved performance headroom within a stable operational zone.

The deterioration index for static SDN starts from 0.224 when $L = 0.5$, peaks temporarily at around 0.285 when $L = 0.9$ due to the presence of a bursty portion in the traffic trace, and then keeps increasing up to 0.302 when $L = 3.5$. In contrast, the SDN+RL performance curve follows an analogous trend at low loads; however, it begins to deviate after $L = 0.9$, when the RL agent starts adapting its weights actively — particularly, it eliminates the burst-triggered peak that occurs under static SDN — and reaches 0.276 at maximum load. From Table IV, it is clear that the reason for the divergence of the curves is related to the classes with lower priority, since the QoS ratio for Clinical reduces from 0.214 for static SDN to 0.106 in SDN+RL, while the ratio for Routine reduces from 0.231 to 0.070. Note that the QoS ratio for Emergency increases slightly, from 0.426 to 0.434.

From the perspective of stability, both configurations satisfy the two-consecutive-points instability criterion throughout the entire load scan process, which confirms that the hospital

TABLE IV
DEGRADATION INDEX DECOMPOSITION AT PEAK LOAD ($L = 3.5$)

Algorithm	DI	Emrg	Crit	Clin	Rtn
SDN + RL	0.276	0.434	0.175	0.106	0.070
SDN	0.302	0.426	0.192	0.214	0.231

fabric architecture is able to operate stably even when the load is up to 3.5 times greater than the nominal ICU load under either policy. On the other hand, the degradation path of the static SDN, which grows steeper and achieves the value of 0.302 at $L = 3.5$, implies that this configuration will reach its stability threshold earlier than SDN+RL as the density of devices and the amount of traffic increase.

V. CONCLUSION AND FUTURE WORK

This paper introduced a priority-aware SDN architecture combined with a tabular Q-learning agent capable of adaptively tuning weights of queue for healthcare IoT. The solution uses empirical replay of ICU traffic, class-aware dynamic routing cost function, and reinforcement learning agent that constantly tunes weights of scheduling across four classes of traffic, namely, Emergency, Critical, Clinical, and Routine. Tested at 16 different load levels from $L = 0.5$ to $L = 3.5$, the proposed SDN + RL approach has shown consistent and significant performance improvements compared to static priority-weighted SDN routing. The peak performance includes 40% drop of overall P95 latency (13.1 ms vs. 21.8 ms), 84% drop of jitter (2.01 ms vs. 12.32 ms), 50.5% improvement in latency of Clinical class and lower Weighted Performance Degradation Index (0.276 vs. 0.302). Importantly, both configurations provided 100% on-time delivery of Emergency and Critical traffic at any load levels, which proves that RL-based adaptive approach does not sacrifice efficiency of life-critical traffic protection for increased overall network efficiency. The RL agent learned optimal scheduling configuration (E:52, C:51, L:50, R:1), demonstrating that online learning approach is able to find non-obvious but effective scheduling configurations for realistic healthcare IoT workloads.

There are several possibilities for further research. Firstly, replacing the tabular Q-learning controller with a Deep Q-Network (DQN) [16] or an actor-critic model would allow using more complex states that would include link utilisation vector, queue statistics and latency history thus allowing more pro-active congestion control. Secondly, testing the proposed framework on real-life SDN hardware using Mininet or testbed infrastructure would provide insight into such parameters as delay between the controller and switch and delay in establishing a new flow. Thirdly, extending the proposed framework to incorporate heterogenous environment with wireless components and edge computing including wearable biomedical sensors [17] [18], mobile sensors, wireless body-area network components would expand its scope to application in future generation hospitals. Finally, including formal methods of verification such as network calculus to provide a provable latency guarantee for the learned policy would be a huge leap towards certification of reinforcement learning-based network management system.

REFERENCES

[1] S. Badotra, D. Nagpal, S. N. Panda, S. Tanwar, and S. Bajaj, "IoT-Enabled Healthcare Network With SDN," in 2020 8th International Conference on Reliability, Infocom Technologies and Optimization (Trends and Future Directions) (ICRITO).

[2] P. Tam, S. Math, and S. Kim, "Priority-Aware Resource Management for Adaptive Service Function Chaining in Real-Time Intelligent IoT Services," *Electronics*, vol. 11, no. 19, p. 2976, 2022.

[3] Y. Su, P. Jiang, H. Chen, and X. Deng, "A QoS-Guaranteed and Congestion-Controlled SDN Routing Strategy for Smart Grid," *Applied Sciences*, vol. 12, no. 15, p. 7629, 2022.

[4] M. Rostami and S. Goli-Bidgoli, "An overview of QoS-aware load balancing techniques in SDN-based IoT networks," *Journal of Cloud Computing*, vol. 13, p. 89, 2024.

[5] B. K. Asingwire, A. Ngenzi, L. Sibomana, and C. Kabiri, "Performance Analysis of IoT-based Healthcare Heterogeneous Delay-sensitive Multi-Server Priority Queuing System," *International Journal of Advanced Computer Science and Applications*, vol. 12, no. 10, 2021.

[6] S. Beitelspacher, K. Besher, and M. Z. Ali, "Sensor Driven Priority Routing of Health Care Data Packet in IoT Network," in *World Forum on Internet of Things (WF-IoT)*, 2020.

[7] T. K. Mannem, N. V. R. V. Gelli, and P. Patel, "Analysis of an IoT-Based SDN Smart Health Monitoring System," in *Microelectronics, Circuits and Systems*, Lecture Notes in Electrical Engineering, pp. 325–334, 2023.

[8] M. S. Parveen and P. T. V. Bhuvaneswari, "A neuro-fuzzy multi-topology adaptive routing framework for QoS-aware healthcare IoT communications," *Scientific Reports*, vol. 16, p. 3330, 2026.

[9] M. Hosseinzadeh, R. Alizadehsani, A. Beheshti, H. Alinejad-Rokny, L. Chen, M. S. Yousefpoor, E. Yousefpoor, M. Altayeb, T. Porntaveetus, and S. Din, "A Q-learning-based QoS-aware multipath routing protocol in IoMT-based wireless body area network," 2024.

[10] M. N. Pathan, M. Muntaha, S. Sharmin, S. Saha, M. A. Uddin, F. N. Nur, and S. Aryal, "Priority based energy and load aware routing algorithms for SDN enabled data center network," *Computer Networks*, vol. 240, p. 110166, 2023.

[11] S. Dilek, K. Irgan, M. Guzel, S. Ozdemir, S. Baydere, and C. Charnsripinyo, "QoS-aware IoT networks and protocols: A comprehensive survey," *International Journal of Communication Systems*, vol. 35, no. 10, p. e5156, 2022.

[12] S. Thangam, H. Rajkumar, A. S. Sannidhi, M. Prajana, and J. J. Kumari, "5G QoS: Enhancing QoS in 5G Networks using Multi-Protocol Label Switching," in 2025 International Conference on Computational Innovations and Engineering Sustainability (ICCIES).

[13] L. Priya P., L. Kandibanda, S. Thangam, J. M. Johnson, and M. Malathi, "Enhanced SDN Controller Placement and Load Balancing for Campus Network Optimization," in 2025 11th International Conference on Communication and Signal Processing (ICCSP).

[14] S. Thangam, M. Gurupriya, A. S. Revanth, D. A. Joel, C. M. Shankar, and I. S. Koushik, "VoIP QoS Refinement through Call Sequencing using Adaptive Jitter Buffer Algorithm," in 2024 15th International Conference on Computing Communication and Networking Technologies (ICCCNT).

[15] S. Misra, S. Pal, N. Ahmed, and A. Mukherjee, "SDN-Controlled Resource-Tailored Analytics for Healthcare IoT System," IEEE Systems Journal, vol. 17, no. 2, pp. 1777–1784.

[16] R. Kulkarni, A. M. Pattanaik, K. Gokul, A. Rajesh, S. S V, and S. S, "Hybrid Machine Learning–based QoE Prediction and SDN-Driven Network Optimization for E-Learning Platforms," in 2026 4th International Conference on Inventive Computing and Informatics (ICICI).

[17] A. M. Jasim and H. Al-Raweshidy, "An Adaptive SDN-Based Load Balancing Method for Edge/Fog-Based Real-Time Healthcare Systems," IEEE Systems Journal, vol. 18, no. 2, pp. 1139–1150.

[18] J. L. Sarkar, R. V, A. Majumder, B. Pati, C. R. Panigrahi, and W. Wang, "I-Health: SDN-Based Fog Architecture for IIoT Applications in Healthcare," IEEE/ACM Transactions on Computational Biology and Bioinformatics, vol. 21, no. 4, pp. 644–651.